

Neural Networks

CS221 - Natural Language Processing

Duy Dang Phu Anh Mai Quoc Bao Luong Huynh Gia Lac
Tran Nguyen Khai

University of Information Technology, VNU-HCM

March 2026

Giảng viên hướng dẫn: TS. Nguyễn Thị Quý

6.6 Training Neural Nets

Nhắc lại: Mục tiêu của quá trình huấn luyện là học các tham số của mô hình sao cho $\hat{y}$ gần với y thực tế nhất có thể.

Các thành phần cần có cho phần huấn luyện:

- Hàm mất mát (**Cross Entropy Loss**)
- Thuật toán tối ưu (**Gradient Descent**)

6.6 Loss Function

Nhắc lại: Gọi y là vector thật (one hot vector), $\hat{y}$ là vector dự đoán của mô hình, K là độ dài của 2 vector, hàm Cross Entropy được định nghĩa:

$$L_{CE}(\hat{y}, y) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

Bài toán dự đoán từ tiếp theo: $K =$ số lượng từ vựng

6.6 Computing the Gradient

Gradient cho hồi quy Softmax:

$$\begin{aligned}\frac{\partial L_{CE}(\hat{y}, y)}{\partial w_{k,i}} &= -(y_k - \hat{y}_k)x_i \\ &= -(y_k - p(y_k = 1|x))x_i \\ &= -(y_k - \frac{\exp(w_k \cdot x + b_k)}{\sum_{j=1}^K \exp(w_j \cdot x + b_j)})\end{aligned}$$

Vấn đề: Chỉ mới đạo hàm cho lớp input (lớp gần với lớp output nhất).

6.6 Computing Graphs

Giả sử một hàm mất mát có dạng:

$$L(a, b, c) = c(a + 2b)$$

Dựa vào các phép $+$, $*$, đặt:

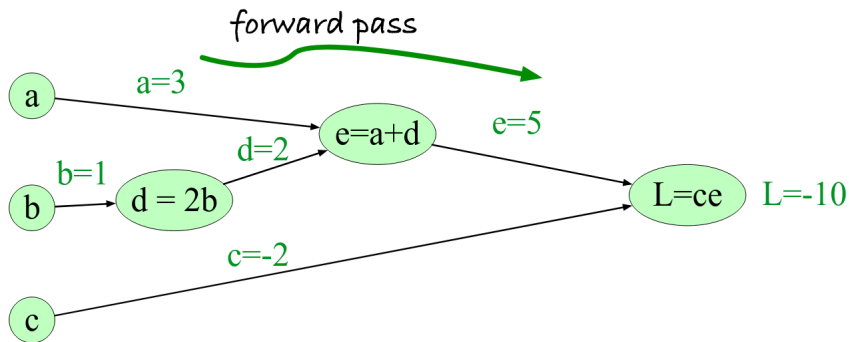
$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

6.6 Computing Graphs

Ta có đồ thị tính toán cho phép tính $c(a + 2b)$:



6.6 Computing Graphs

Giả sử hàm mất mát là $L = c * (a + 2b)$, ta muốn lấy đạo hàm $\frac{dL}{db}$ như thế nào?

6.6 Computing Graphs

Giả sử hàm mất mát là $L = c * (a + 2b)$, ta muốn lấy đạo hàm $\frac{dL}{db}$ như thế nào?

Ta sẽ dùng **chain rule**, tính tích các đạo hàm thành phần.

6.6 Computing Gradients

Giả sử hàm mất mát là $L = c * (a + 2b)$, ta muốn lấy đạo hàm $\frac{dL}{db}$ như thế nào?

Ta sẽ dùng **chain rule**, tính tích các đạo hàm thành phần.

Công thức **chain rule** cho đạo hàm $f(x) = u(v(x))$ theo x :

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}.$$

6.6 Computing Gradients

Giả sử hàm mất mát là $L = c * (a + 2b)$, ta muốn lấy đạo hàm $\frac{dL}{db}$ như thế nào?

Ta sẽ dùng **chain rule**, tính tích các đạo hàm thành phần.

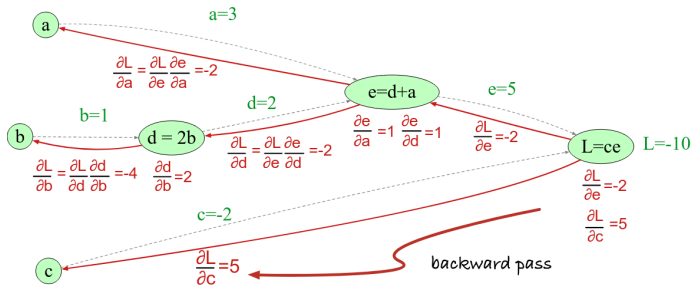
Công thức **chain rule** cho đạo hàm $f(x) = u(v(x))$ theo x :

$$\frac{df}{dx} = \frac{du}{dv} = \frac{du}{dv} \cdot \frac{dv}{dx}.$$

Ví dụ cho hàm $u(v(w(x)))$:

$$\frac{du}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$$

6.6 Computing Graphs



Dựa vào đồ thị trên, ta có thể tính đạo hàm của $L = c * (a + 2b)$ theo các thành phần b :

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial d} \cdot \frac{\partial d}{\partial b} = 2c$$

6.6 Backward differentiation for a neural network

Giả sử ta có một mạng neural:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = ReLU(z^{[1]})$$

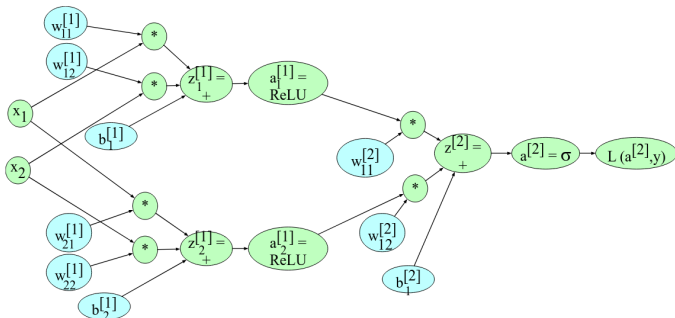
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

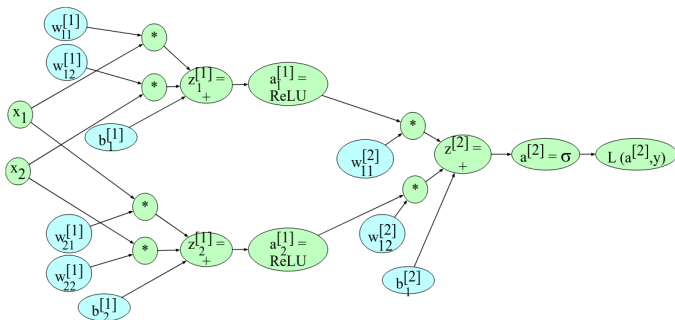
$$\hat{y} = a^{[2]}$$

6.6 Backward differentiation for a neural network

Ta có sơ đồ tính toán của mạng neural.



6.6 Backward differentiation for a neural network



Dựa vào đồ thị và chain rule, đạo hàm của L theo $W_{21}^{[1]}$:

$$\frac{\partial L}{\partial W_{21}^{[1]}} = \frac{\partial L}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a_2^{[1]}} \cdot \frac{\partial a_2^{[1]}}{\partial z_2^{[1]}} \cdot \frac{\partial z_2^{[1]}}{\partial W_{21}^{[1]}}$$

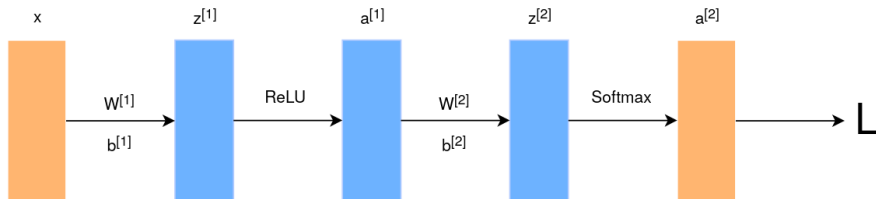
6.6 Backward differentiation for a neural network

Phân tích:

- $\frac{\partial L}{\partial z^{[2]}} = a^{[2]} - y$
- $\frac{\partial z^{[2]}}{\partial a_2^{[1]}} = W_{12}^{[2]}$
- $\frac{\partial a_2^{[1]}}{\partial z_2^{[1]}}$: Đạo hàm của **ReLU**.
- $\frac{\partial z_2^{[1]}}{\partial W_{21}^{[1]}} = x_2$

Tương tự, dựa vào đồ thị và phân tích **chain rule**, ta có thể tính đạo hàm của L theo các tham số của mô hình.

6.6 Backward differentiation for a neural network

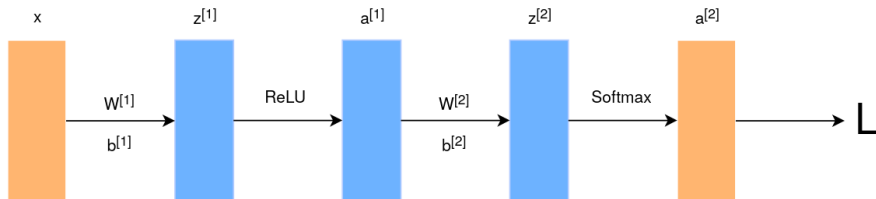


Trong thực tế, hàm Loss sẽ được đạo hàm theo một vector hay một ma trận.

Đạo hàm của hàm Loss theo $z^{[1]}$:

$$\frac{\partial L}{\partial z^{[1]}} = \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial L}{\partial z^{[2]}}$$

6.6 Backward differentiation for a neural network



Tham số của mô hình được cập nhật dựa trên các gradient tìm được.
Ví dụ: cập nhật tham số $W^{[1]}$ bằng thuật toán **Gradient Descent**:

$$W^{[1]} = W^{[1]} - \eta * \frac{\partial L}{\partial W^{[1]}}$$

Practical Issues

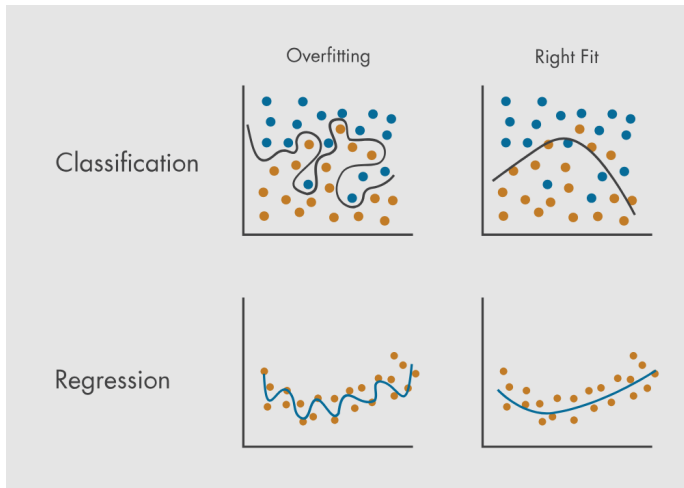
- Tại sao mô hình sau khi huấn luyện thường không hoạt động tốt ngay lập tức?
- Các thách thức chính:
 - **Overfitting** (Quá khớp): Mô hình học thuộc lòng dữ liệu huấn luyện thay vì tìm ra quy luật tổng quát.
 - **Learning Rate** (Tốc độ học): Quyết định biên độ cập nhật trọng số.

Overfitting

- **Định nghĩa:** Mô hình học thuộc lòng dữ liệu huấn luyện (noise) thay vì học quy luật chung.
- **Dấu hiệu:** Loss trên tập Train rất thấp, nhưng Loss trên tập Validation/Test lại rất cao.
- **Nguyên nhân:** Số lượng tham số quá lớn so với lượng thông tin thực tế trong dữ liệu.

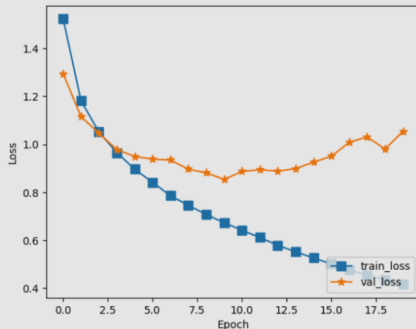
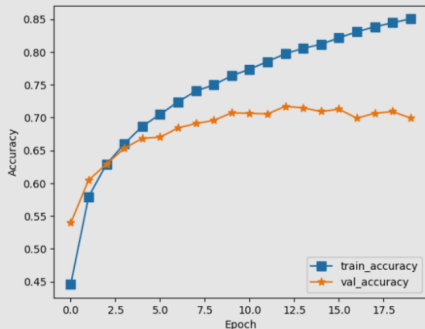
Overfitting

Minh họa hiện tượng Overfitting



Overfitting

Hiện tượng Overfitting xảy ra trong quá trình Training



Overfitting

Các kĩ thuật chống Overfitting phổ biến:

- **Regularization (L1, L2):** Thêm thành phần phạt (penalty) vào hàm Loss để giới hạn độ lớn của trọng số.
- **Dropout:** Ngẫu nhiên "tắt" các neuron để ngăn chặn sự phụ thuộc quá mức giữa các lớp.
- **Early Stopping:** Dừng quá trình huấn luyện khi lỗi kiểm định ngừng giảm hoặc bắt đầu tăng.

Reguralization

- **L1 Regularization:**

$$L = L_0 + \lambda \sum_{i=1}^n |w_i|$$

→ Tạo ra sự **thưa thớt (Sparsity)**: Triệt tiêu các trọng số không quan trọng về 0 (tự động chọn lọc đặc trưng).

- **L2 Regularization:**

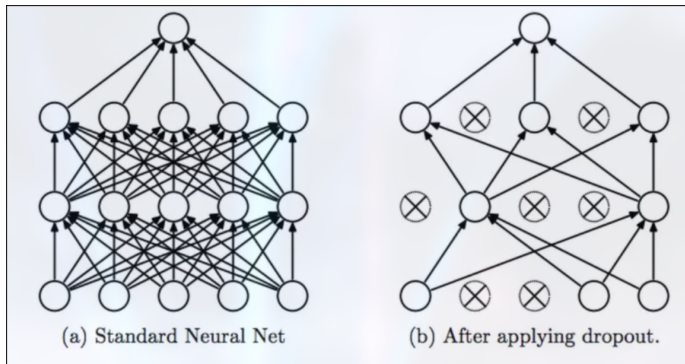
$$L = L_0 + \lambda \sum_{i=1}^n w_i^2$$

→ **Weight Decay**: Giảm dần giá trị trọng số nhưng không triệt tiêu hẳn, giúp mô hình ổn định và bền vững hơn.

- **Hệ số λ** : Điều chỉnh cường độ phạt. λ lớn khiến mô hình đơn giản hơn; λ quá nhỏ sẽ mất tác dụng chống Overfitting.

Dropout

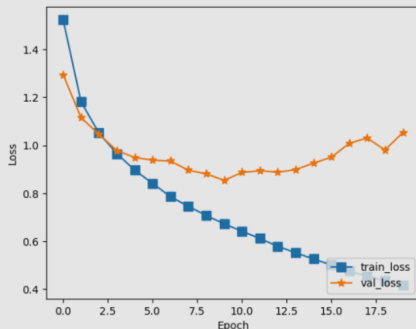
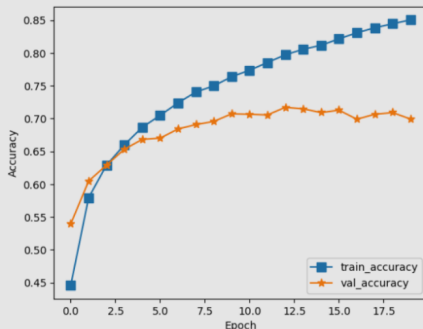
- **Cách hoạt động:** Trong mỗi bước huấn luyện, ngẫu nhiên "tắt" (gán bằng 0) một tỉ lệ phần trăm các neuron, thường là 20% – 50%.



- **Cách hoạt động:** Trong mỗi bước huấn luyện, ngẫu nhiên "tắt" (gán bằng 0) một tỉ lệ phần trăm các neuron, thường là 20% – 50%.
- **Tại sao hiệu quả?**
 - Ngăn chặn các neuron "hợp tác" quá chặt chẽ để ghi nhớ dữ liệu (giảm Co-adaptation).
 - Ép mỗi neuron phải tự học những đặc trưng hữu ích một cách độc lập.
- **Lưu ý quan trọng:**
 - Chỉ sử dụng Dropout trong quá trình **Train**.
 - Bắt buộc phải **tắt** Dropout khi thực hiện **Test**.

Early Stopping

- **Định nghĩa:** Kỹ thuật giúp ngăn chặn hiện tượng Overfitting bằng cách giám sát hiệu suất mô hình trên tập Validation
- **Nguyên lý:**
 - Dừng quá trình huấn luyện khi Loss trên tập Validation ngừng giảm hoặc bắt đầu tăng
 - Lưu lại bộ trọng số của mạng tại thời điểm **Validation Loss thấp nhất**.



Early Stopping

- **Định nghĩa:** Kỹ thuật giúp ngăn chặn hiện tượng Overfitting bằng cách giám sát hiệu suất mô hình trên tập kiểm định Validation
- **Nguyên lý:**
 - Dừng quá trình huấn luyện khi Loss trên tập Validation ngừng giảm hoặc bắt đầu tăng
 - Lưu lại bộ trọng số của mạng tại thời điểm **Validation Loss thấp nhất**.
- **Siêu tham số kiên nhẫn (Patience):** Số vòng lặp cho phép mô hình tiếp tục huấn luyện dù không có sự cải thiện trước khi dừng hẳn.
- **Lợi ích:**
 - Ngăn chặn Overfitting một cách tự động.
 - Tiết kiệm đáng kể thời gian và tài nguyên tính toán.

Learning rate

- **Định nghĩa:** η là một siêu tham số (hyperparameter) quan trọng, quyết định độ lớn của bước nhảy khi cập nhật trọng số mô hình trong quá trình cực tiểu hóa hàm mất mát

$$W = W - \eta * \frac{\partial L}{\partial W}$$

Learning rate

- **Định nghĩa:** η là một siêu tham số (hyperparameter) quan trọng, quyết định độ lớn của bước nhảy khi cập nhật trọng số mô hình trong quá trình cực tiểu hóa hàm mất mát
- **Tác động của giá trị:**
 - **Quá lớn:** Gây dao động mạnh, dễ vượt qua điểm tối ưu hoặc khiến mô hình không thể hội tụ.
 - **Quá nhỏ:** Huấn luyện rất chậm, tốn tài nguyên và dễ bị kẹt tại các cực tiểu địa phương.
- **Chiến lược điều chỉnh tối ưu:**
 - **Learning Rate Schedulers:** Giảm dần η theo thời gian để mô hình ổn định hơn khi tiến gần điểm hội tụ.
 - **Adaptive Learning Rate:** Các thuật toán tự động điều chỉnh η cho từng tham số riêng biệt (như **Adam**, **RMSprop**).